

COMS W4156 Advanced Software Engineering (ASE)

November 9, 2023



Initial Feedback on First Assessment

You should be able to see the rubric in courseworks now

We will finish grading as soon as we can. Mentors grade their own team members. Those that have been graded so far look good, all 90+ (out of 100)

I've read all the responses to part 0A/0B. The feedback from those of you who chose to answer 0B was quite helpful, thank you

But some of your answers to 0A concerned me



Concerns from First Assessment



A small number of students will receive 0 on the first assessment because they ignored the instructions for part 0A, which were discussed in class several times as well as appearing in the assessment itself: *"It is very important that you explain the scores you give to each student, including yourself: If you do not provide meaningful explanations, then your response will be considered incomplete and you will receive 0 overall on the assessment."*

Numbers with no explanation is not a meaningful explanation. *"Each team member did equal amount of work and contributed equally"* [and nothing further] is not a meaningful explanation

Even if you report that all the team members did equal work, it's important for you to tell us who did what. This is useful to the teaching staff for determining how well you understand your own project. If you don't know who did what, not even what you did yourself, it's hard to believe that you are contributing your fair share. This is also taken into consideration in 'differential grading'

Agenda

1. Origin of Design Patterns
2. Observer
3. Mediator
4. Facade
5. Adapter
6. MVC
7. Strategy



How do we choose a design for a particular kind of software development problem?

We look at what everyone else uses for that kind of problem or similar problems

- *Originality is overrated*
- *Imitation is the sincerest form of not being stupid*



Our (not yet written) software almost certainly corresponds to some “pattern”

Architectural Patterns



Before there were patterns in software, before there was any software, there were patterns in cities and individual buildings

Christopher Alexander codified what architects and civil engineers had long known in his books about patterns

On building architecture: “Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.”

-- Christopher Alexander

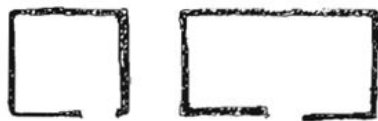


196 CORNER DOORS*

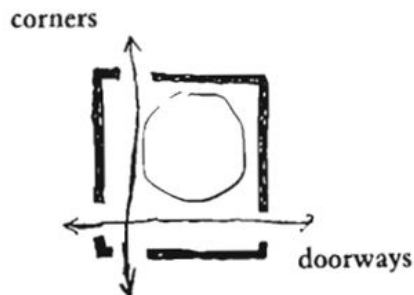
The success of a room depends to a great extent on the position of the doors. If the doors create a pattern of movement which destroys the places in the room, the room will never allow people to be comfortable.



First there is the case of a room with a single door. In general, it is best if this door is in a corner. When it is in the middle of a wall, it almost always creates a pattern of movement which breaks the room in two, destroys the center, and leaves no single area which is large enough to use. The one common exception to this rule is the case of a room which is rather long and narrow. In this case it makes good sense to enter from the middle of one of the long sides, since this creates two areas, both roughly square, and therefore large enough to be useful. This kind of central door is especially useful when the room has two partly separate functions, which fall naturally into its two halves.



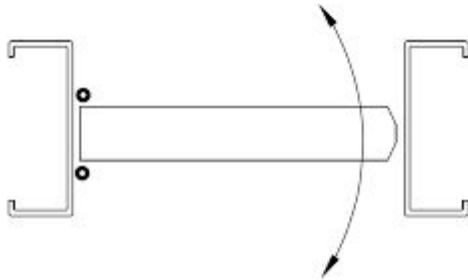
Rooms with one door.



Except in very large rooms, a door only rarely makes sense in the middle of a wall. It does in an entrance room, for instance, because this room gets its character essentially from the door. But in most rooms, especially small ones, put the doors as near the corners of the room as possible. If the room has two doors, and people move through it, keep both doors at one end of the room.

If we were designing buildings instead of software, would Professor Alexander's pattern solve our problem of where to put doors?

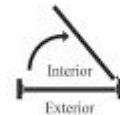
Not entirely - which way should the door(s) swing?



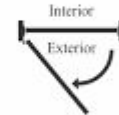
Swing Door Handling



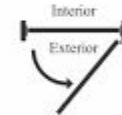
Left Hand
Opens To Interior



Right Hand
Opens To Interior

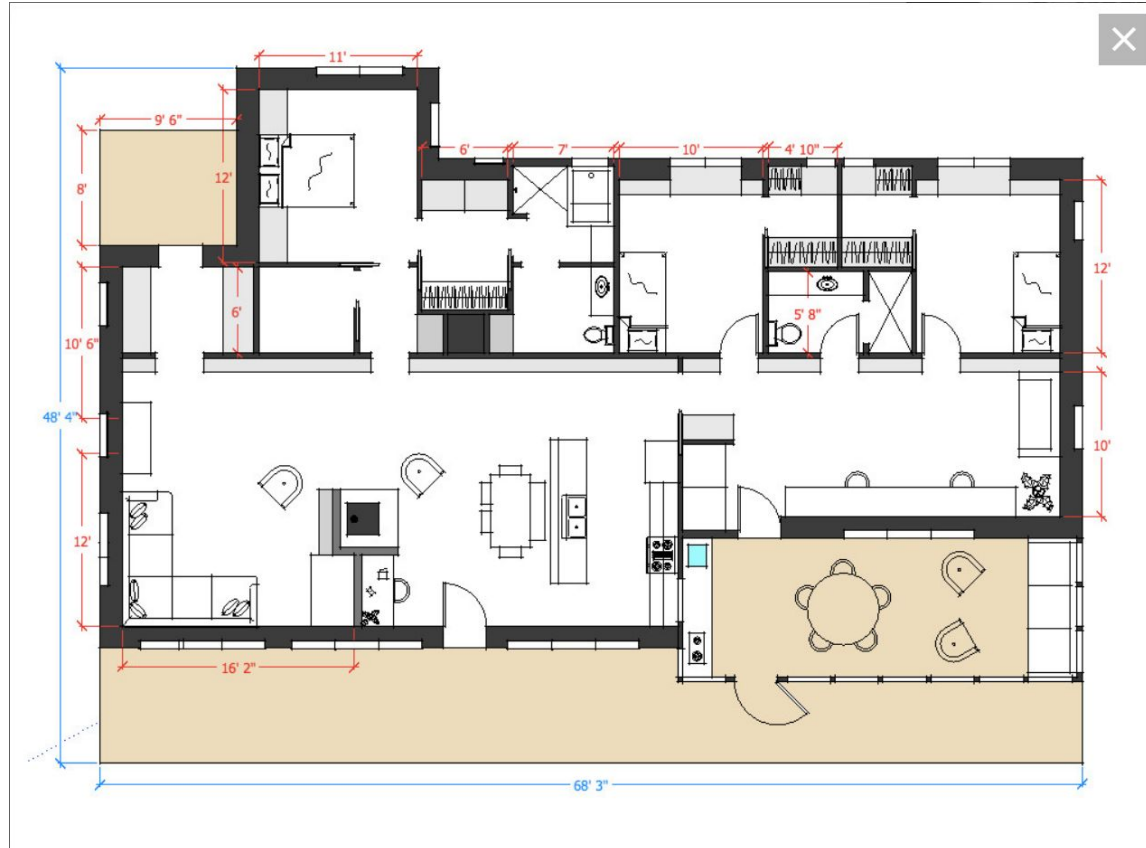


Left Hand Reverse
Opens To Exterior



Right Hand Reverse
Opens To Exterior

A small house attempting to use Alexander's "A Pattern Language"



Architecture patterns do not provide solutions to every aspect of the problem



paraveina

6 years ago

Speaking from experience, that door to the porch will be slammed against your WD every time it opens. I suggest it should either open out, or if you still want to be able to do a screen, swing the other way or move it down slightly.

👍 Like ❤️ Save

chelwa thanked paraveina



Alexander's Architectural Pattern Elements

1. Name of the pattern
2. Purpose of the pattern = what problem it solves
3. How to use the pattern to solve the problem
4. Constraints to consider in solution



These elements work just as well for software as for buildings

- Reuse well-understood and tested architectures and designs
- Save time and don't reinvent the wheel
- Communication language among software engineers

What is the Difference between Architecture and Design?

Software engineers need version control (like [git](#)) because software changes over time, possibly very long periods of time

Non-trivial software is not implemented all-at-once, and software with real users changes over long periods of time to fix bugs, add features, and adapt to changing environment and context. Refactoring along the way may substantially re-organize system structure

Design principles, such as the [SOLID principles](#), are centered on change, trying to make the code more resilient to change - more flexible, extensible, and maintainable (micro level)

[Architecture](#) is fundamental system structure that rarely changes (macro level) - client/server, N-tier, broker, pub/sub, etc.

? Should we cover software architecture in more depth ?



Gang of Four book

[Design patterns](#) are general reusable solutions to commonly occurring problems, typically defined by a description or template rather than a library or framework with reusable code

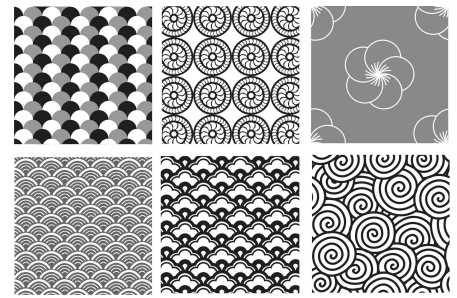
Initial set of 23 design patterns presented in GoF [book](#), heavily influenced by Christopher Alexander's [architectural patterns for buildings and cities](#)

The book gives examples in C++ and Smalltalk ([Java wasn't available yet](#))

See “15 years later” interview with GoF authors [here](#)



Behavioral vs. Structural vs. Creational



- [Creational patterns](#) provide class instantiation and object creation mechanisms that increase flexibility and reuse of existing code
- [Structural patterns](#) explain how to assemble objects and classes into larger structures, while keeping the structures flexible and efficient
- [Behavioral patterns](#) realize common communication paradigms and the assignment of responsibilities between objects

Singleton design pattern is creational, Observer, Mediator and Strategy are behavioral, Facade and Adapter are structural

Agenda

1. Origin of Design Patterns
2. **Observer**
3. Mediator
4. Facade
5. Adapter
6. MVC
7. Strategy



Observer Pattern

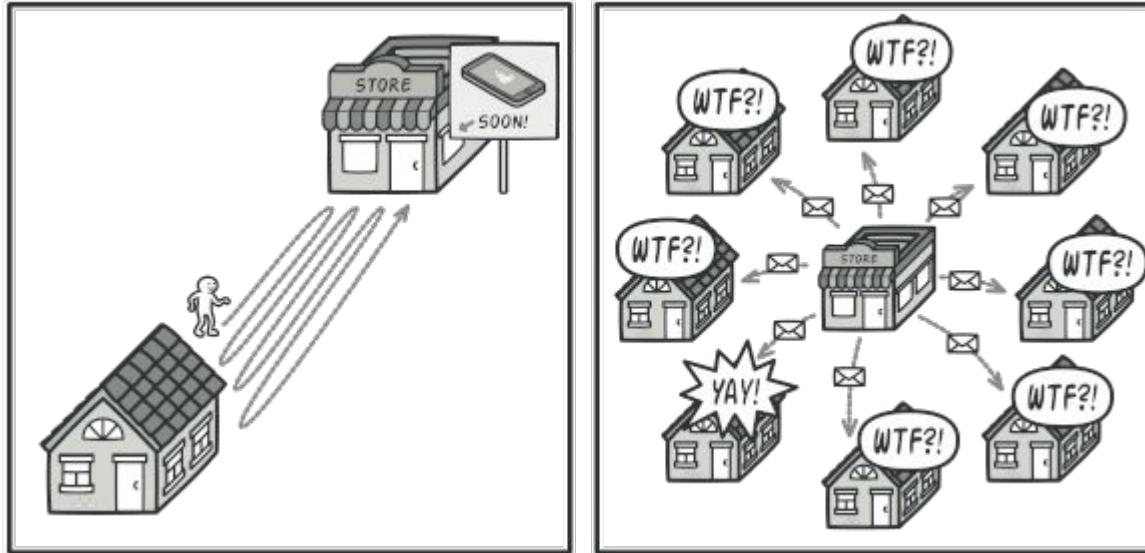
Behavioral design pattern that lets you define a subscription mechanism to notify multiple objects (called subscribers) about any events that happen to the object they're observing (called publisher or subject)

Analogous to [publish/subscribe architecture](#), but at object instead of system granularity

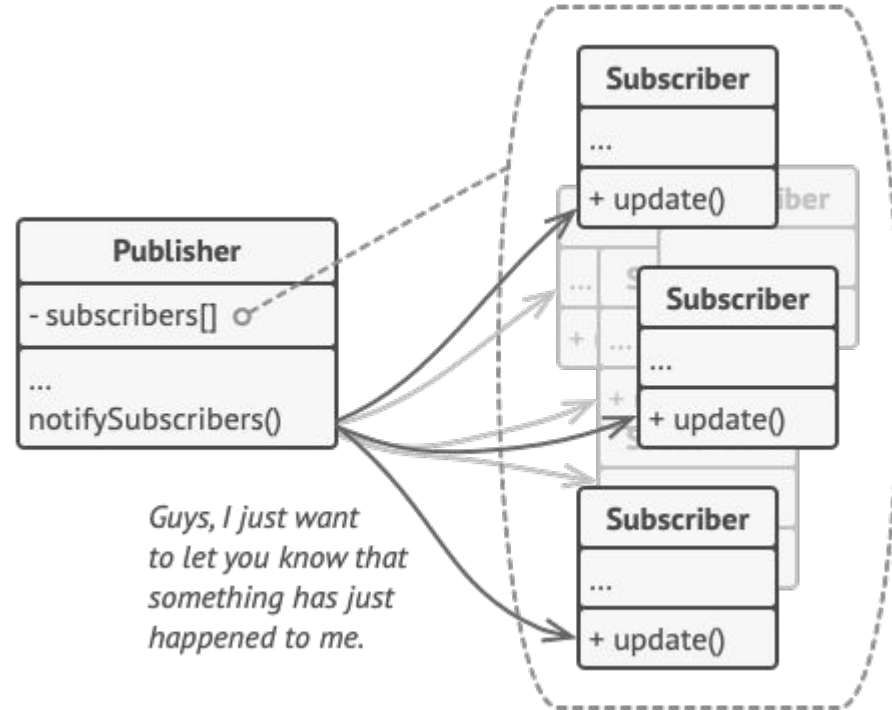
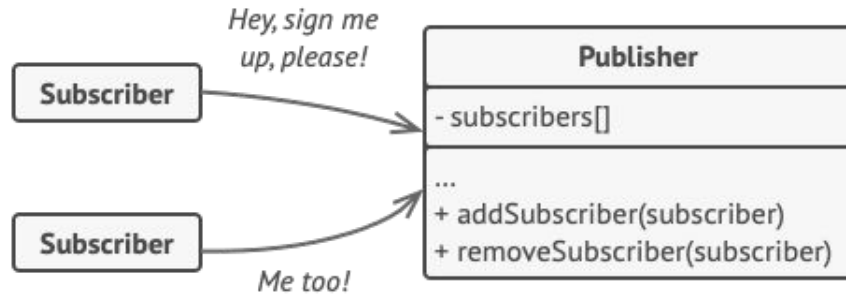
Behavioral patterns realize common communication paradigms and the assignment of responsibilities between objects



Problem that Observer Addresses

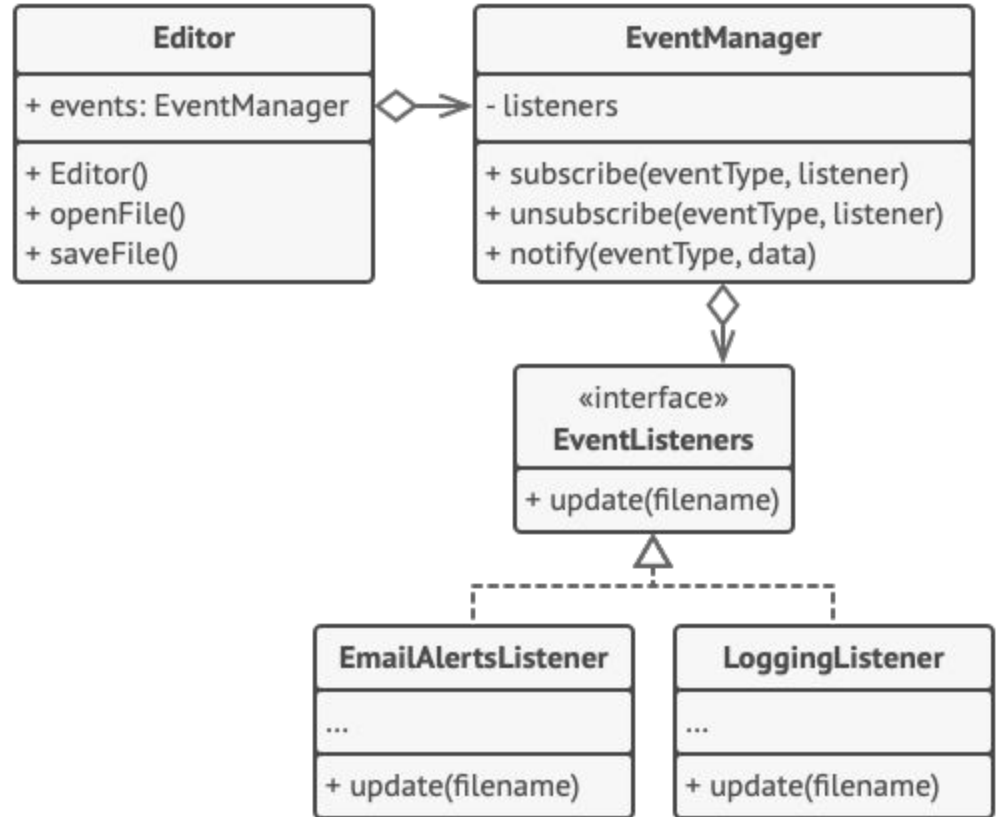


Observer Solution



Example Observer

Note: Diagrams on this page and previous page (and upcoming pages) are [UML class diagrams](#) . The arrows between boxes have [special meanings](#)



? More UML later ?

When to Use Observer

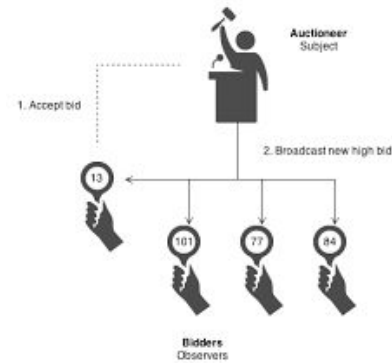
Use when changes to the state of one object may require changing other objects, particularly when the actual set of objects is unknown beforehand or changes dynamically - The Observer pattern lets any object that implements the subscriber interface subscribe for event notifications from publisher objects, e.g., to hook up custom code (callbacks) via custom subscriber classes

Use when some objects in your program must observe others, but only for a limited time or in specific cases - the subscription list is dynamic, so subscribers can join or leave the list whenever they need to



Refactoring to use Observer

1. Break your business logic into two parts: core functionality will act as the publisher, the rest will turn into a set of subscriber classes
2. Declare subscriber interface with, at least, an update method that takes “context” as parameter(s) - perhaps a reference to the publisher itself so subscribers can ask it more specifically for what they want when they are notified
3. Declare publisher interface with a pair of methods for adding and removing a subscriber object
4. Decide where to put the actual subscription list and subscription methods, maybe an abstract class extended by concrete publishers or a separate object referenced by concrete publishers (composition)
5. Create concrete publisher classes that notify all their subscribers whenever something important happens
6. Implement update notification methods in concrete subscriber classes and code to register and deregister with publishers as needed

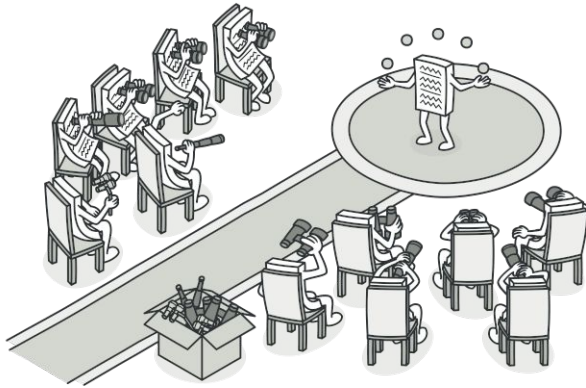


Observer Pattern Code Examples

[C++](#)

[Java](#)

[Typescript](#)



stopped here

Agenda

1. Origin of Design Patterns
2. Observer
3. Mediator
4. Facade
5. Adapter
6. MVC
7. Strategy

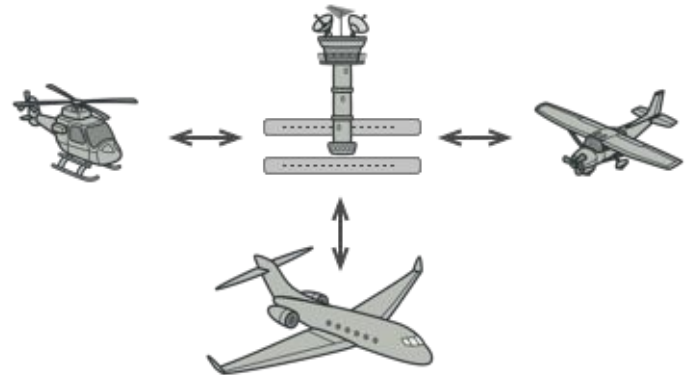


Mediator Pattern vs. Observer Pattern

Behavioral design pattern that lets you eliminate mutual (two-way) dependencies among objects, restricts direct communications between the objects and forces them to instead depend only on a single mediator object

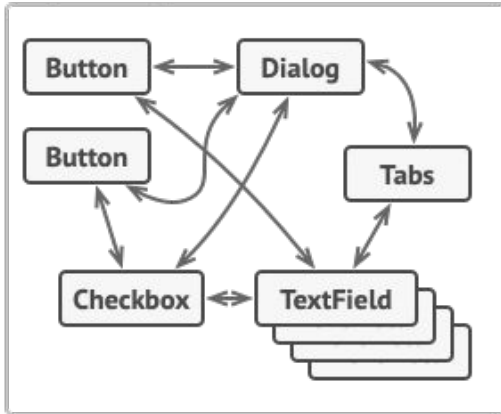
In contrast, observer dependencies are dynamic and one-way

Could use observer to implement mediator. ? How ?

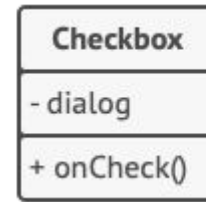
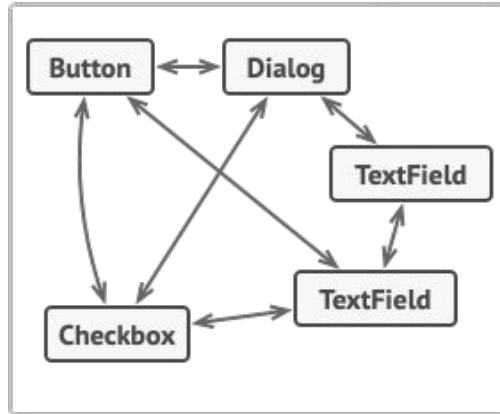


Problem that Mediator Addresses

Profile Dialog



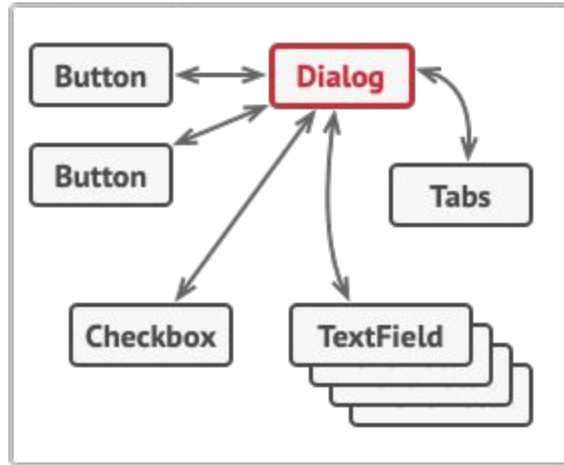
Login Dialog



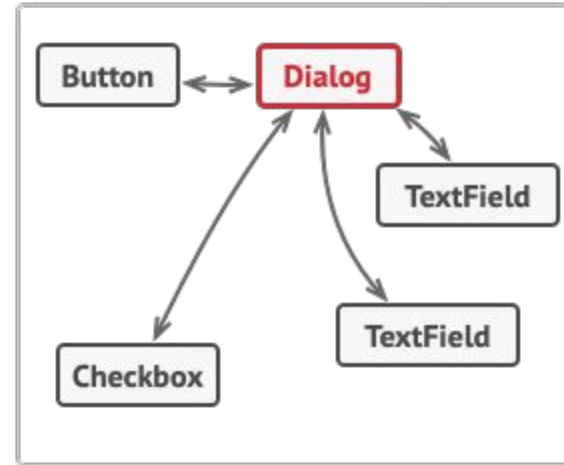
```
if (dialog.name == "profile_form")
    // ...
if (dialog.name == "login_form")
    // ...
```

Mediator Solution

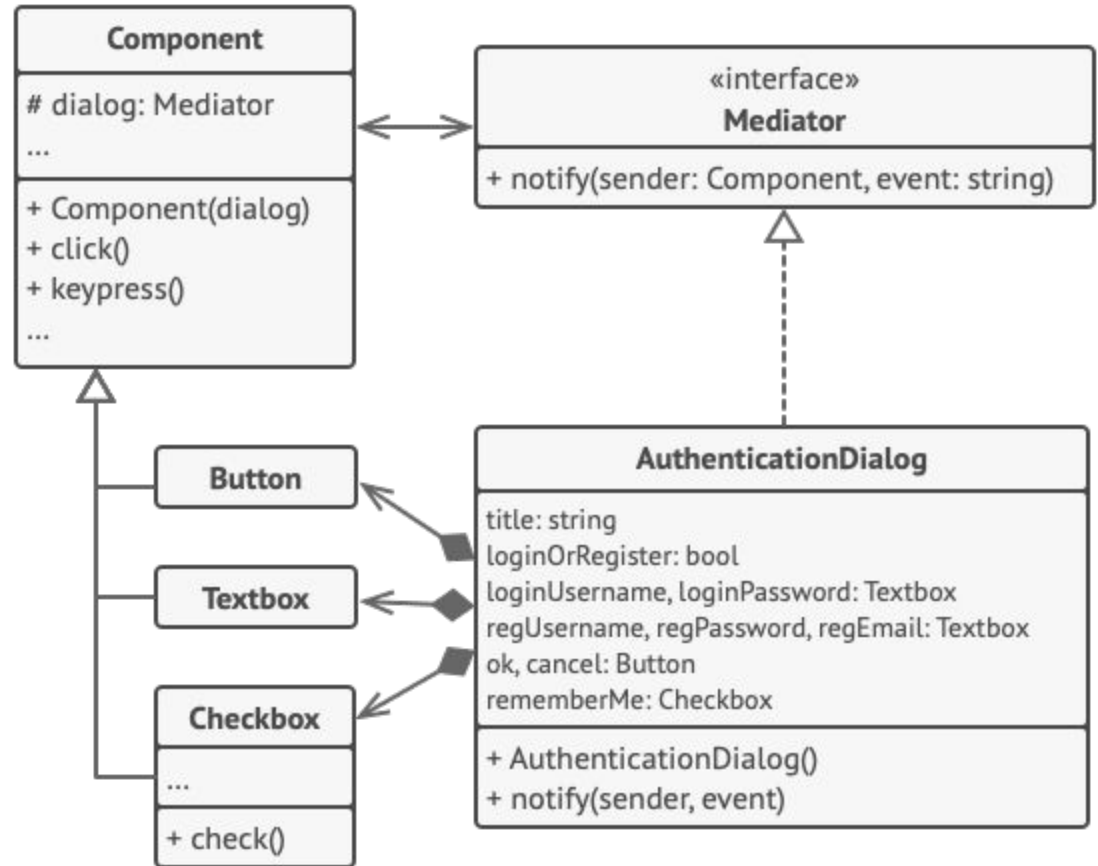
Profile Dialog



LogIn Dialog



Mediator Example



When to Use Mediator

Use when some classes are tightly coupled to several other classes, so they are hard to change - the Mediator pattern lets you extract all the relationships between classes into a separate class, isolating any changes to a specific component

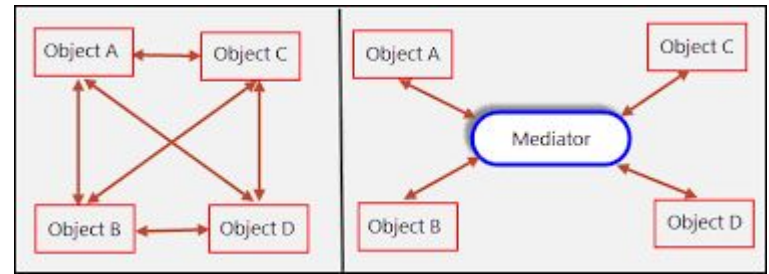
Use when you can't reuse a component in a different program or in a different way because it's too dependent on your other components - After you apply Mediator, individual components become unaware of the other components so can reuse by providing a new mediator class



shutterstock.com • 1976767568

Refactoring to Use Mediator

1. Identify a group of tightly coupled classes
2. Declare a mediator interface with the desired communication protocol between mediators and those classes, in many cases a single method for receiving notifications is sufficient
3. Implement a concrete mediator class, which should store references to all components it manages
4. Components should also store a reference to the mediator object, e.g., pass the mediator object as an argument to the component's constructor
5. Change the components' code so that they call the mediator's notification method instead of methods on other components
6. Extract the code that involves calling other components into the mediator class, and execute this code whenever the mediator receives notifications from that component

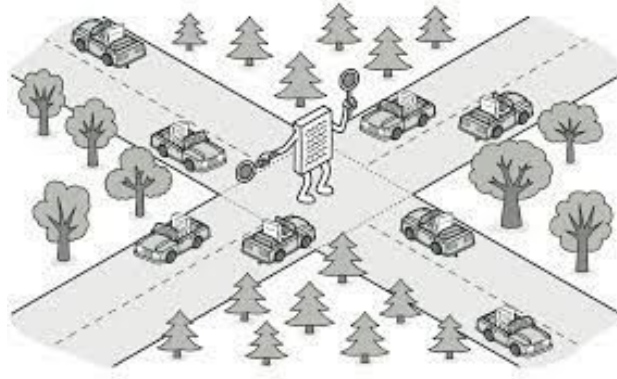


Mediator Pattern Code Examples

[C++](#)

[Java](#)

[Typescript](#)



Agenda

1. Origin of Design Patterns
2. Observer
3. Mediator
4. **Facade**
5. Adapter
6. MVC
7. Strategy



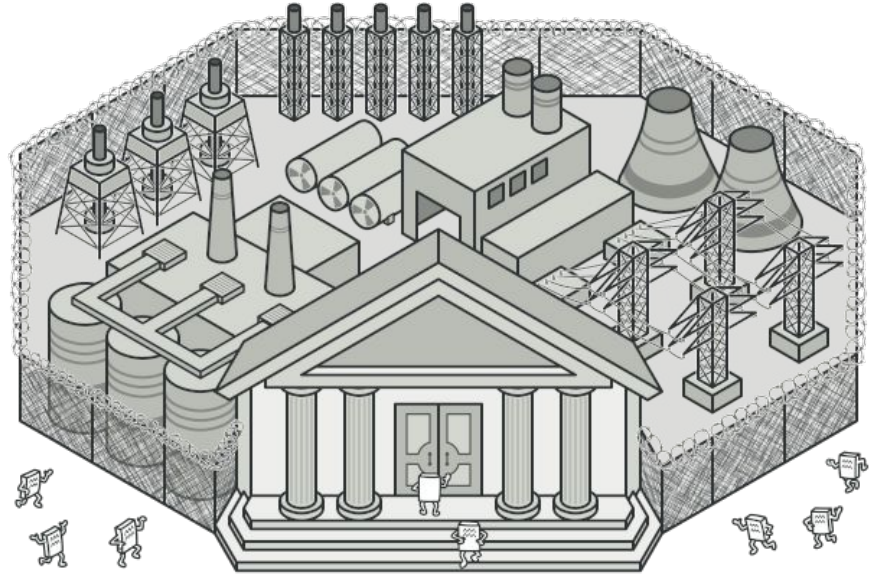
Facade Pattern is Like a Mediator for a Subsystem

Facade is a structural design pattern that provides a simplified interface to a library, framework, or other complex set of classes

Structural patterns explain how to assemble objects and classes into larger structures, while keeping the structures flexible and efficient

Facade and Mediator have similar jobs: they try to organize collaboration between lots of tightly coupled classes

Facade defines a simplified interface to a subsystem of objects, but objects *within* the subsystem can still communicate directly



Problem that Facade Addresses

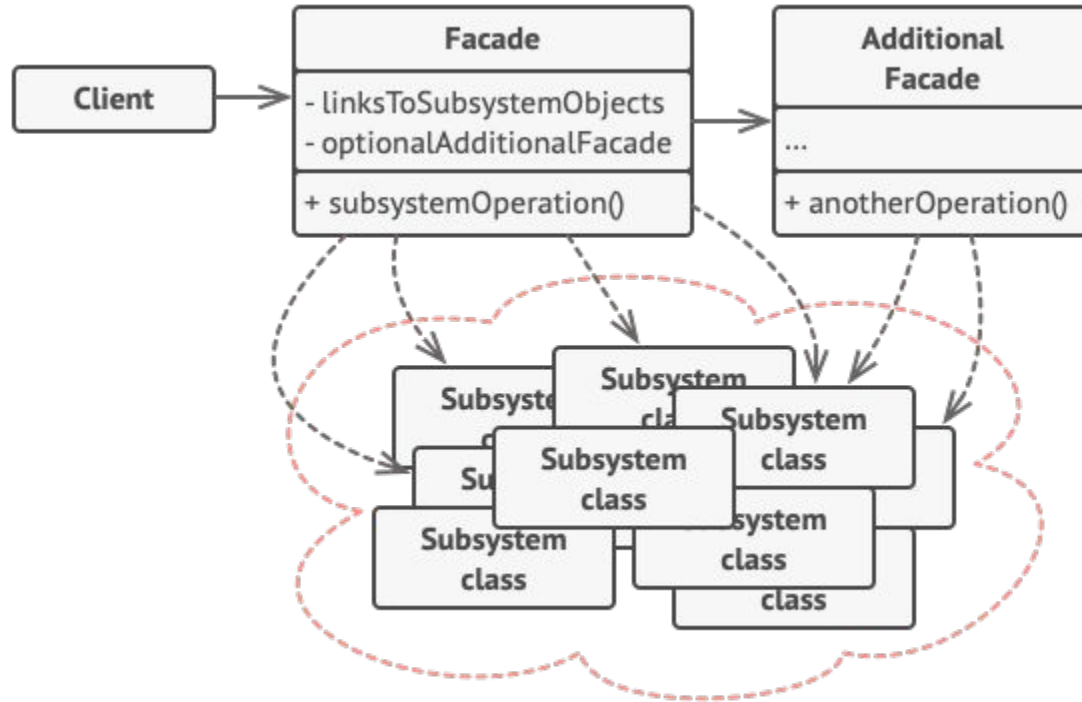


Imagine your code needs to work with a *subset* of some fancy third-party library or framework, you're not a power-user who needs full control of "everything"

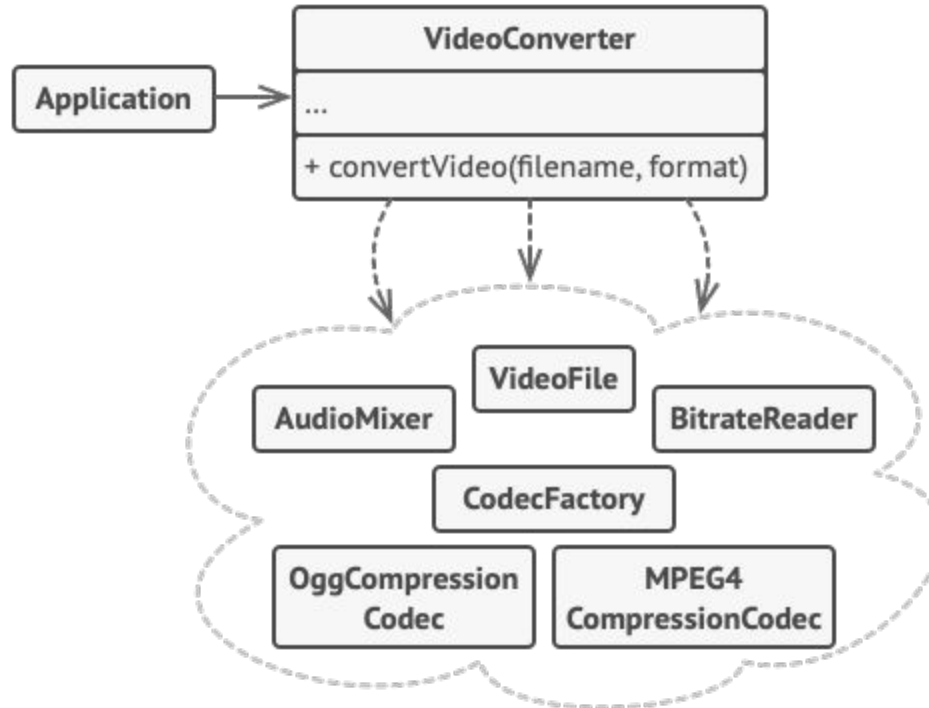
You'd still need to initialize all the third-party objects you use (and possibly others you don't), keep track of dependencies among those objects, execute method calls on different objects in the correct order, supply data in proper format that might be different for different objects, and so on

Your business logic would become tightly coupled to the implementation details of 3rd-party classes

Facade Solution



Facade Example



When to Use Facade

Use the Facade pattern when you need to have a limited but straightforward interface to a complex subsystem

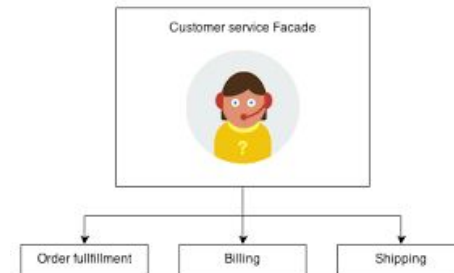
Notice the Facade pattern is presented as the developer adding a facade to *someone else's* library or framework, but alternatively the subsystem could come with multiple facades to choose from

Similarly to applying Mediator, your components become unaware of the individual components within the subsystem so can replace with a different subsystem implementing the same interface



Refactoring to use Facade

1. Check whether it's possible to provide a simpler interface than what an existing subsystem already provides - you're on the right track if this interface makes the client code independent from many of the subsystem's classes.
2. Declare and implement this interface in a new facade class. The facade should redirect the calls from the client code to appropriate objects of the subsystem, and is responsible for initializing the subsystem and managing its life cycle
3. To get the full benefit from the pattern, make all the client code communicate with the subsystem only via the facade, protecting the client from any changes in the subsystem code
4. If the facade becomes too big, consider extracting part of its behavior to a new, refined facade class

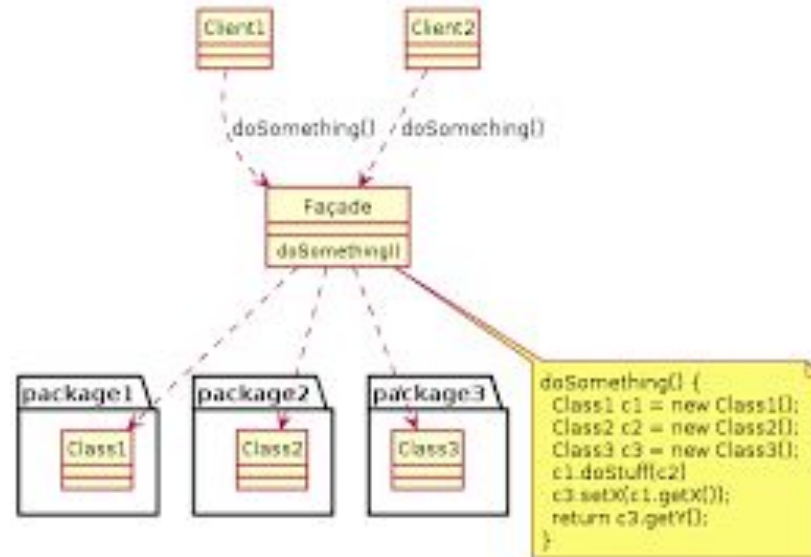


Facade Pattern Code Examples

[C++](#)

[Java](#)

[Typescript](#)



Agenda

1. Origin of Design Patterns
2. Observer
3. Mediator
4. Facade
5. **Adapter**
6. MVC
7. Strategy

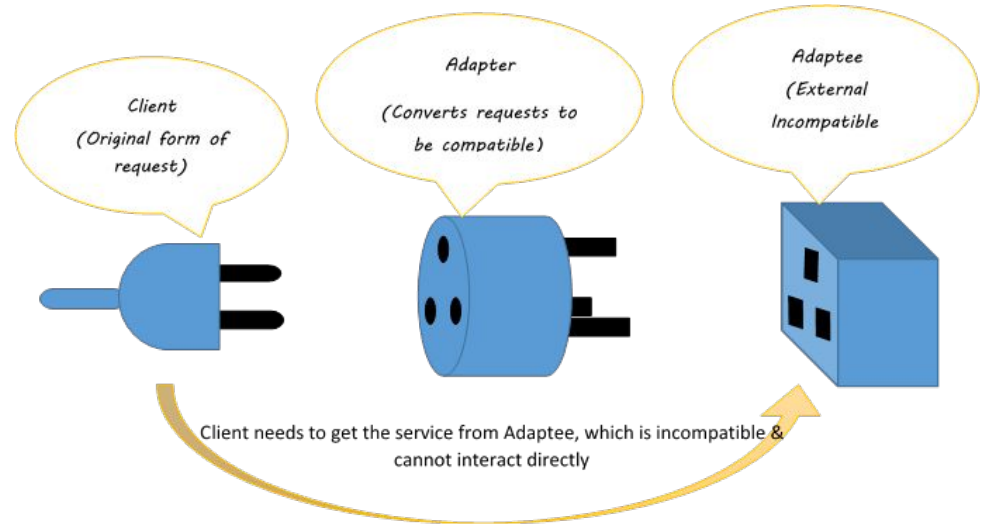
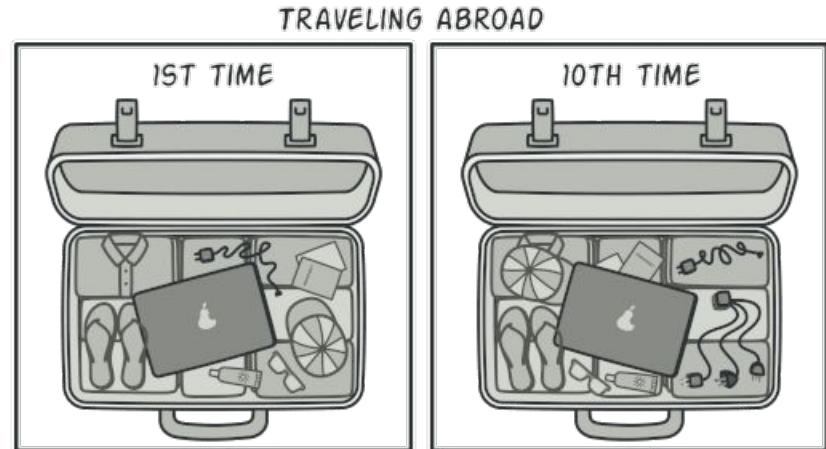


Figure 1-Adapter Pattern Concept

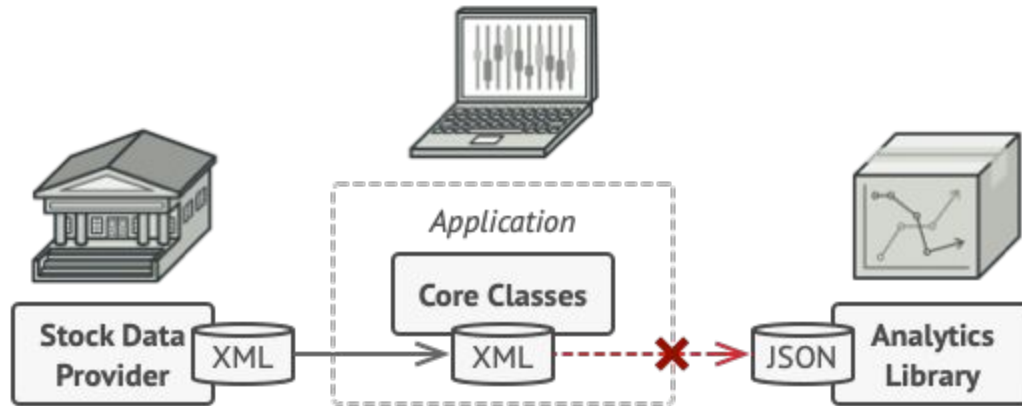
Adapter Pattern Converts Interface instead of Simplifying

Structural design pattern that allows objects with incompatible interfaces to collaborate

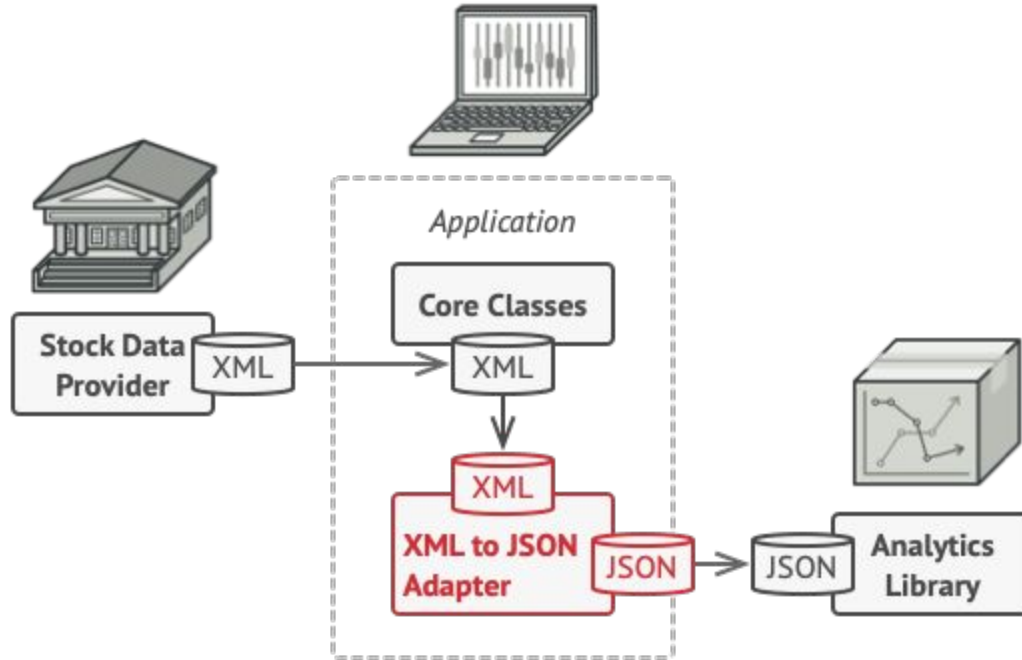
Adapter (often called wrapper) usually wraps one class of objects or literally one singleton object, not a whole subsystem like Facade



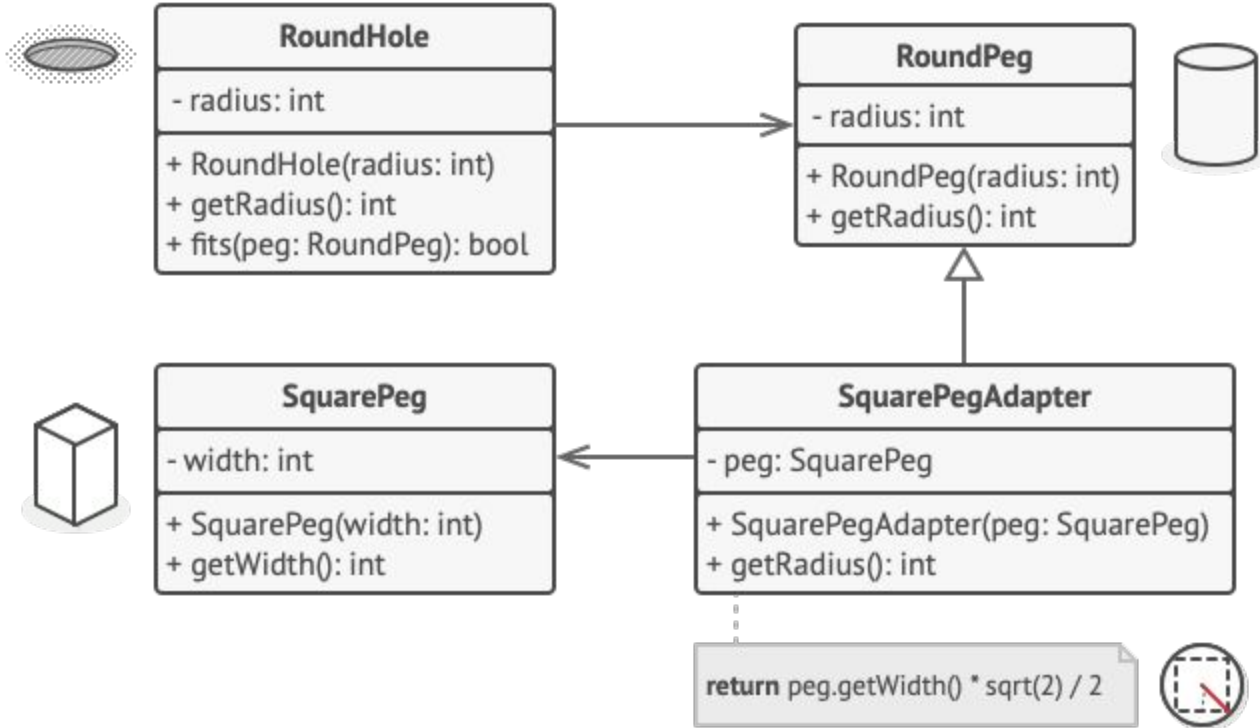
Problem that Adapter Addresses



Adapter Solution



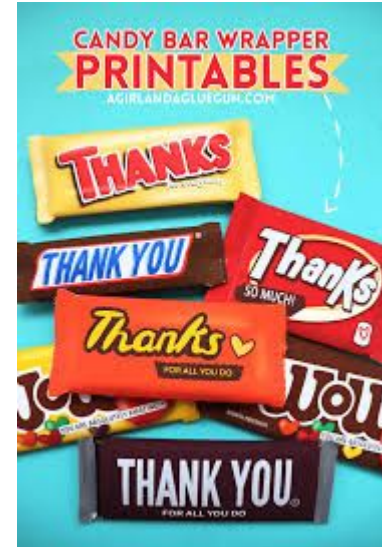
Adapter Example



When to Use Adapter

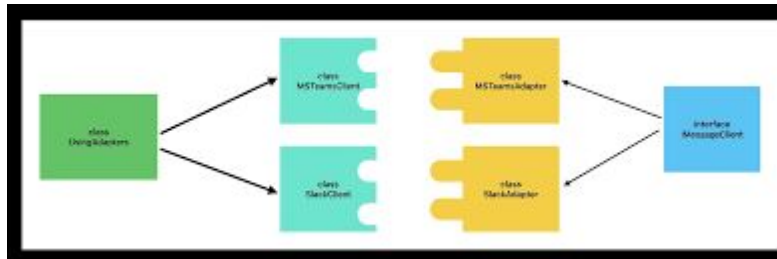
Use when you want to use some existing class, but its interface isn't compatible with the rest of your code - the Adapter pattern creates a middle-layer class to translate between your code and a legacy class, 3rd-party class or any other class that cannot be changed

Another application is when you want to reuse several existing subclasses that lack some common functionality that can't be added to the superclass - rather than duplicate the missing functionality in the child classes, wrap inside an Adapter that implements that functionality once



Refactoring to Use Adapter

1. Declare the client interface for how clients should communicate with the service
2. Create an adapter class that follows the client interface, initially with empty methods
3. Add a field to the adapter class to store a reference to the service object and initialize this field via the adapter's constructor (alternatively could pass the relevant service object to the adapter as an argument to each method)
4. Implement all methods of the client interface in the adapter class, handling only the interface or data format conversion and delegating most of the real work to the service object
5. Clients then use the adapter via the client interface



Adapter Pattern Code Examples

[C++](#)

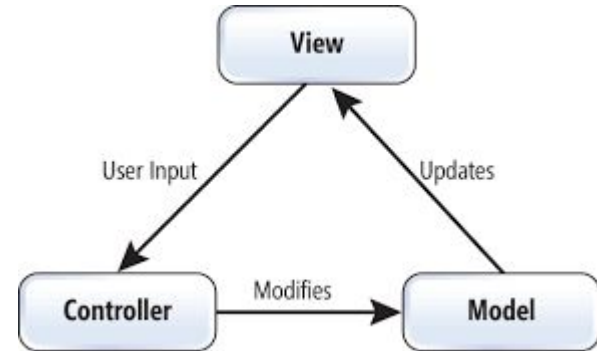
[Java](#)

[Typescript](#)



Agenda

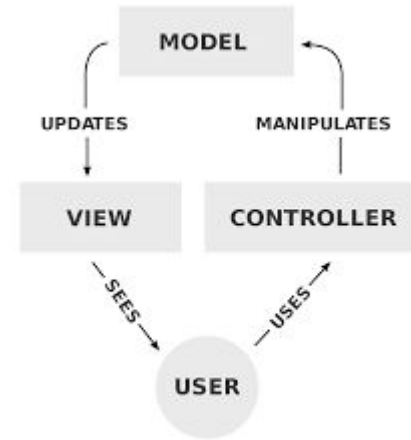
1. Origin of Design Patterns
2. Observer
3. Mediator
4. Facade
5. Adapter
6. **MVC**
7. Strategy



Model-View-Controller (MVC)

MVC is usually described in terms of user-facing app, but also applies to non-UI software

- Model contains the core functionality and data,
- View displays the information to the user,
- Controller handles user input entered through the view and directs the model to 'do something'



MVC Analogy

In a restaurant, the Chef is the Model.
It's job is to take orders (requests)
from the Waiter and create food for
the Waiter to deliver to the Customer



MVC Analogy

A Waiter is a Controller. Waiters take in orders (requests), take them back to the Chef (Model), and then deliver the proper meal to Customers (Users)

The Table is the View. It just holds whatever meal the Waiter brings back from the Chef

There can be multiple controllers and views for the same model



Model

Includes business logic that manipulates state and does most of the work as well as CRUD operations for persistent state (application or domain data). Ideally a fully functional application program (with no GUI)

- Not possible to alter data except through the model
- Provides APIs to views and controllers that enable the outside world to communicate with it
- Responds to requests for information about its state (usually from the view, possibly through the controller)
- Responds to instructions to do something or change state (from the controller)



View and/or Controller may include some persistent state, such as saved layout and preferences, but the Model *owns* the domain data

View

GUI output: all CSS & HTML code goes here, updates when model changes, acts as passive observer that does not affect the model

- May be multiple views of same data, such as web, tablet and phone versions of an app
- Push - view registers callbacks with model, model notifies of changes
- Pull - view polls model for changes

Technically, MVC views do not necessarily involve a user interface, a view could be a representation 'visible' to other code



Controller

GUI input: accepts inputs from user, translates user actions on view into operations on model, decides what model should do

- May be structured as intermediary between view and model or as intermediary between user and view (renders/displays view for user to see)
- May be multiple controllers for same model

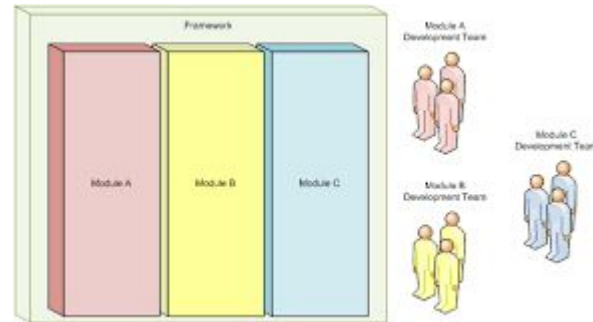
“Routes” = extension of MVC for REST APIs



“Separation of Concerns”

Important software engineering concept that you will keep hearing about

- Model is loosely coupled with the view and controller, knows nothing about either ⇒ *modularity*
- View and controller may be tightly coupled
- In OO languages usually implemented as two or three different classes that can be developed separately



Example: music player app

- View - playlists, current song, graphics, sound, video, etc., with some elements that accept user selection, action, data entry
- Controller - interprets the user actions applied to view and tells model what to do
- Model - stores content, knows how to play songs, shuffle, rip, and so on

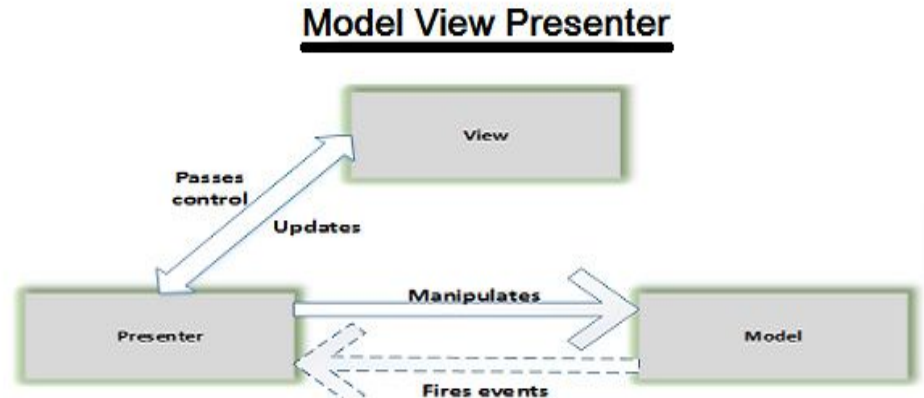
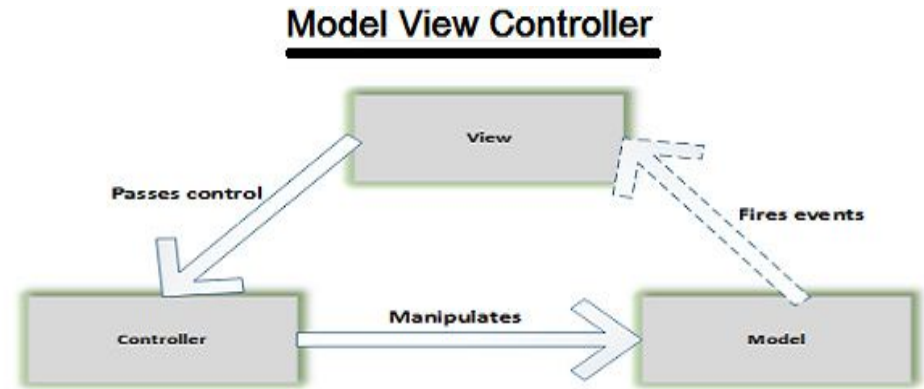


MVC Variants

Controversy over what belongs in the model

The model might be *only* the data (with possibly some bookkeeping methods) and the business logic resides in the controller (which better matches [N-tier architecture](#))

But then neither the model nor the controller is a full application by itself



How to decide?

Business logic belongs in the model when the data, and thus the model, is only part of one application

But the developer may not know in advance whether the model might eventually be used for other applications ([software sustainability](#))



Business logic belongs in the controller when the data, and thus the model, is shared across many applications

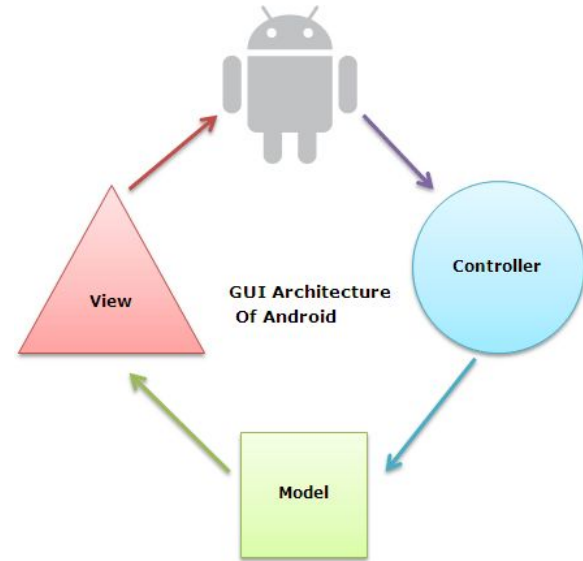
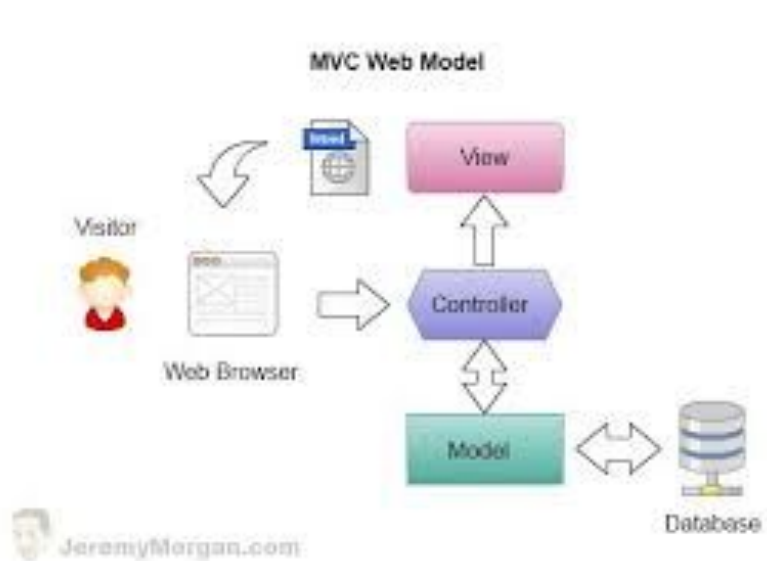
Different controller (and presumably different view or set of views) for each application

Enables replacing model while retaining controller (and views)

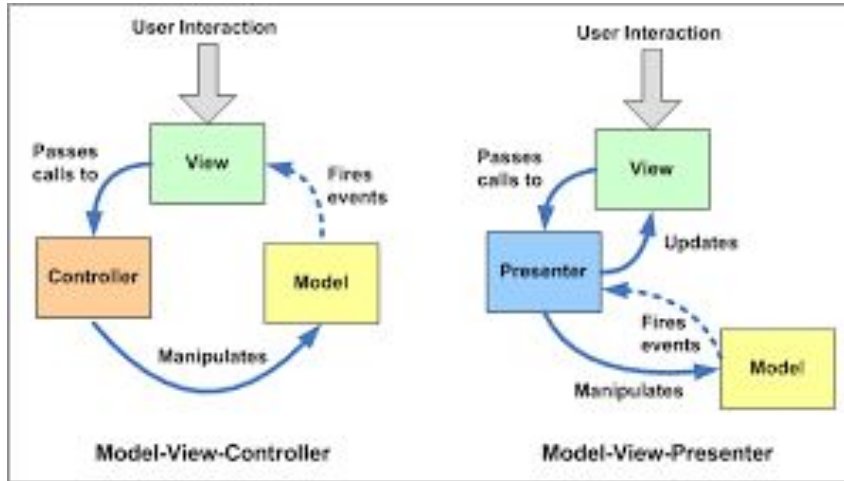
Is MVC really a design pattern?

It's not in the GoF book

But it works and everyone uses it (or something like it)



Is MVC a design pattern?



MVC - uses Observer Pattern to passively update view

MVP - uses Observer Pattern to passively update presenter, which in turn actively updates the view

([There are other variants](#))

MVC/P is a *Compound* Design Pattern

Observer Pattern + Strategy Pattern

Observer Pattern updates the view to display the model, possibly multiple different views for the same model, e.g., desktop vs. mobile browser

Strategy Pattern supports different ways (“strategies”) to leverage the model, e.g., multiple apps using the same service



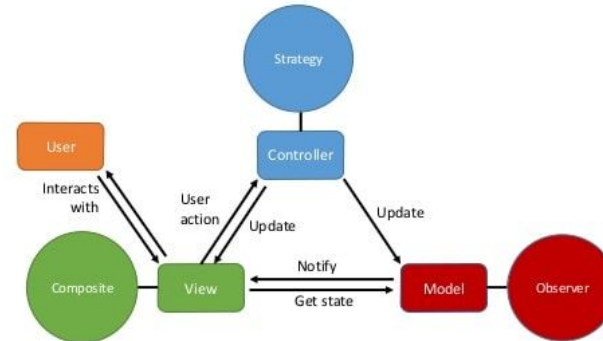
MVC Code Examples

[C++](#)

[Java](#)

[Node.js](#)

Traditional MVC & associated design patterns



1. R. Buencamino: Learning the Model-View-Controller Design Patterns in iOS. Lynda.com, 2015

Agenda

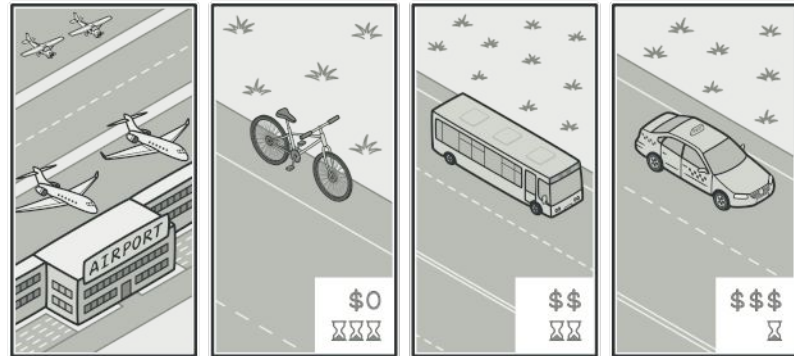
1. Origin of Design Patterns
2. Observer
3. Mediator
4. Facade
5. Adapter
6. MVC
7. Strategy



What's the Strategy Pattern?

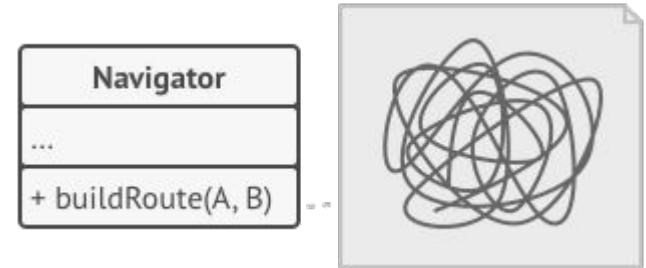
Behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable

May allow swapping algorithms at runtime - e.g., in MVC/P an administrator might switch on the fly between privileged and regular user interfaces with different controller/presenter not just different view (e.g., [mice fake student](#))

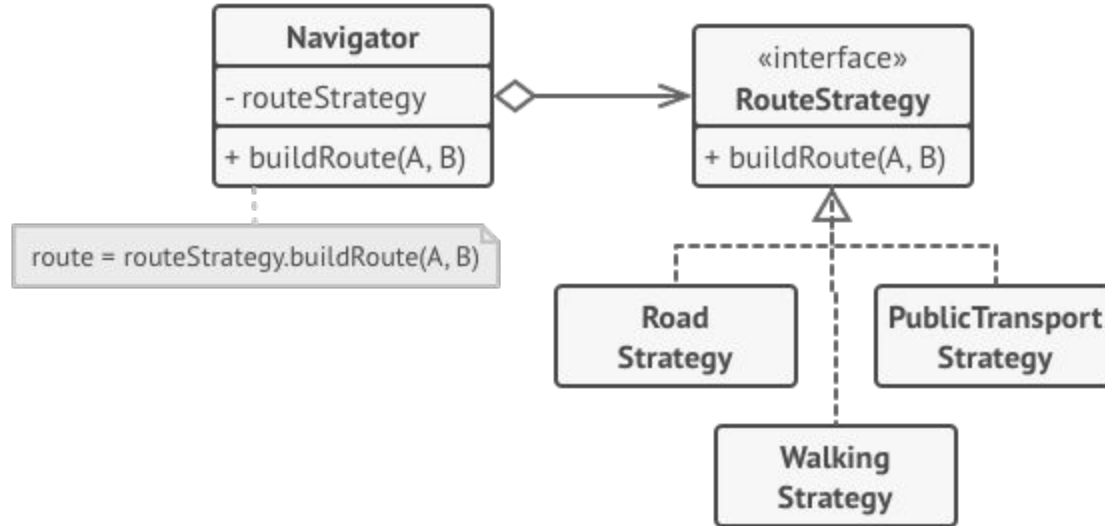


Problem that Strategy Addresses

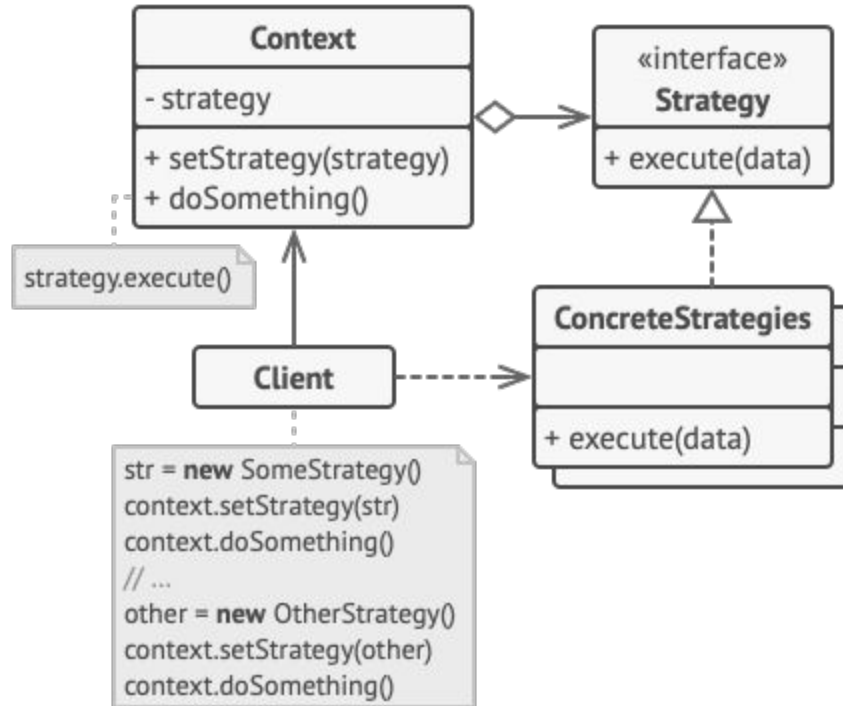
Google maps automatic route planning



Strategy Solution



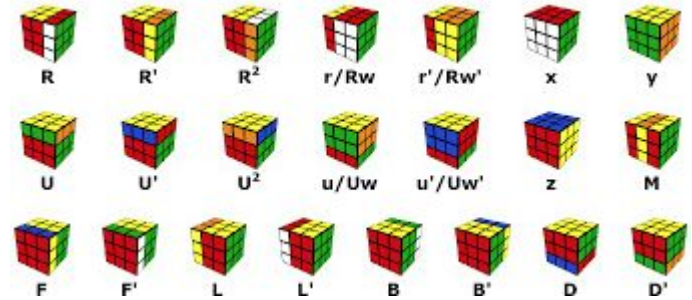
Strategy Example



When to Use Strategy

Use when you want to use different variants of an algorithm within an object and be able to switch from one algorithm to another - the Strategy pattern lets you indirectly alter the object's behavior at runtime by associating it with different sub-objects that perform specific sub-tasks in different ways

Use when you have a lot of similar classes that only differ in the way they execute some behavior - extract the varying behavior into a separate class hierarchy and combine the original classes into one, thereby reducing duplicate code



Use the Strategy pattern when your class has a massive conditional operator that switches between different variants of the same algorithm

Refactoring to Use Strategy



1. In the context class, identify an algorithm that's prone to frequent changes or has many different variants
2. Declare the strategy interface common to all variants of the algorithm.
3. Extract all algorithms into their own classes that implement the strategy interface
4. In the context class, add a field for storing a reference to a strategy object and provide a setter for replacing values of that field
5. The context class also needs to supply an interface for the strategy object to access the context's data
6. Clients of the context must associate it with a suitable strategy that matches the way they expect the context to perform its primary job

Strategy Pattern Code Examples

[C++](#)

[Java](#)

[Typescript](#)



Many More Design Patterns

? Should we cover more of the design patterns ?

The Sacred Elements of the Faith

the holy
origins

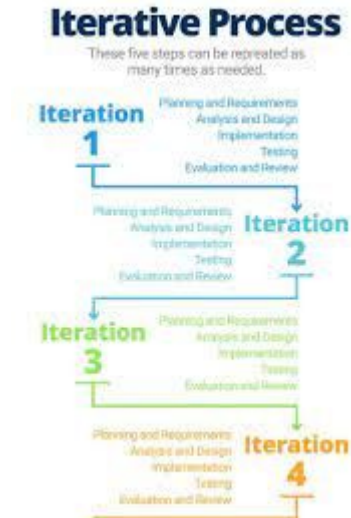
the holy
structures

the holy
behaviors

107 FM Factory Method							139 A Adapter
117 PT Prototype	127 S Singleton				223 CR Chain of Responsibility	163 CP Composite	175 D Decorator
87 AF Abstract Factory	325 TM Template Method	233 CD Command	273 MD Mediator	293 O Observer	243 IN Interpreter	207 PX Proxy	185 FA Façade
97 BU Builder	315 SR Strategy	283 MM Memento	305 ST State	257 IT Iterator	331 V Visitor	195 FL Flyweight	151 BR Bridge

Upcoming Assignments

- [Second Iteration](#) due Thursday, November 30, by 11:59pm
- [Second Iteration Demo](#) due Thursday, December 7, by 11:59pm



Next Week

Tuesday: Advertisement for my spring course and finish Design Patterns (assuming we don't finish today)

Thursday: Software Deployment with GCP demo

Ask Me Anything